



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 1
по курсу «Разработка параллельных и
распределенных программ»

*Распараллеливание алгоритма вычисления
произведения двух матриц*

Студент: Федуков А. А.

Группа: ИУ9-52Б

Преподаватель: Царев А. С.

Москва, 2025 г.

Содержание

Постановка задачи	2
Практическая реализация	3
Код программы	4
Вывод программы	11
Вывод	11
Характеристики компьютера	12

Постановка задачи

Даны две квадратные матрицы A и B размера $n \times n$. Требуется выполнить следующие этапы.

1. Последовательная реализация (стандартный алгоритм)

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}, \quad i, j = 1, \dots, n.$$

Реализовать умножение матриц в виде тройного цикла (i, j, k). Замерить время выполнения и сохранить полученную матрицу C_{ref} как эталон.

2. Порядок обхода элементов Реализовать альтернативную последовательную версию, в которой элементы матрицы C вычисляются не по строкам (внешние циклы i затем j), а по столбцам (внешние циклы j затем i). Сравнить времена двух версий и объяснить наблюдаемую разницу (кэш-эффекты, порядок доступа к A и B).

3. Подбор размера матриц Подобрать n так, чтобы время выполнения на вашей машине было разумным (не менее нескольких секунд — не более нескольких минут для одного прогона). Привести выбранное значение n и обоснование.

4. Параллельная реализация (разбиение на подматрицы) - Разделить результирующую матрицу C условно на r примерно равных прямоугольных подматриц (например, по строкам, по столбцам или в сетке $r \times s$, где $r \cdot s =$ число потоков). - Соответственно разделить матрицу A на группы строк, а матрицу B — на груп-

пы столбцов, чтобы каждая задача имела локальные данные для вычисления своей подматрицы `C_block`. - Для каждого потока (задачи) вычислять только элементы своей подматрицы, записывая их в уникальную область `C` (чтобы избежать гонок данных). Не использовать библиотеки для умножения матриц. - Организовать синхронизацию: основной поток, породив рабочие потоки, должен ждать их завершения (например, `join` для `std::thread` или эквивалент в вашей среде). При необходимости использовать механизмы сигнализации/ожидания, но блокировки для записи в разные области `C` не нужны, если разбиение непересекающееся.

5. Эксперименты и измерения - Выполнить вычисления для разного числа потоков (например, 1, 2, 4, 8, ...), для нескольких разбиений (по строкам, по столбцам, по блокам). Для каждого варианта замерить время и построить таблицу/график ускорения (`speedup`) относительно последовательного эталона. - Выполнить несколько прогонов и усреднить время, указать дисперсию. - После каждого параллельного прогона сравнить полученную матрицу `C_parallel` с эталонной `C_ref` (покомпонентно, с учетом допустимой погрешности для вещественных чисел, например $|a-b| < 1e-9$). Зафиксировать корректность.

6. Отчёт В отчёте привести: - описание реализованных алгоритмов (последовательных и параллельных), - параметры эксперимента (`n`, число потоков, разбиения), - измеренные времена и расчёты ускорения/эффективности, - анализ влияния порядка обхода и разбиения на производительность, - замечания по проблемам синхронизации и использованным средствам измерения (высокоточный таймер).

Примечания и рекомендации: - Используйте тип `double` для вычислений. - Для измерений применять высокоточный таймер (`std::chrono::high_resolution_clock` или эквивалент). - Избегайте глобального конкурентного обновления одних и тех же элементов `C` — каждое ядро должно записывать в свою область. - Для проверки корректности лучше сравнивать с последовательной эталонной версией, выполненной до параллельных экспериментов.

Практическая реализация

Код программы

Листинг 1: Исходный код программы

```
1 use rand::random_range;
2
3 use std::hash::{DefaultHasher, Hash, Hasher};
4 use std::io::Write;
5 use std::sync::{
6     Arc,
7     atomic::{AtomicBool, Ordering},
8     mpsc::channel,
9 };
10 use std::thread;
11 use std::time::{Duration, Instant};
12
13 const MAX_N: usize = 1000; // Max size of the matrix
14 type Elem = i32; // Type of the matrix elements
15
16 #[derive(Clone)]
17 struct Matrix {
18     rows: usize,
19     cols: usize,
20     data: Vec<Elem>,
21     data_hash: u64,
22 }
23
24 impl Hash for Matrix {
25     fn hash<H: Hasher>(&self, state: &mut H) {
26         for value in &self.data {
27             value.hash(state);
28         }
29     }
30 }
31
32 impl Matrix {
33     /// Initialize a new matrix with given dimensions
34     fn new(rows: usize, cols: usize) -> Self {
35         assert!(rows <= MAX_N && cols <= MAX_N);
36         Matrix {
37             rows,
38             cols,
39             data: vec![0; rows * cols],
```

```

40         data_hash: 0,
41     }
42 }
43
44 /// Fill the matrix with random values
45 fn fill(&mut self) {
46     for i in 0..self.rows {
47         for j in 0..self.cols {
48             self.data[i * self.cols + j] = random_range(0..100);
49         }
50     }
51 }
52
53 /// Set the value at position (i, j)
54 fn set(&mut self, i: usize, j: usize, value: Elem) {
55     assert!(i < self.rows && j < self.cols);
56     self.data[i * self.cols + j] = value;
57 }
58
59 /// Get the value at position (i, j)
60 fn get(&self, i: usize, j: usize) -> Elem {
61     assert!(i < self.rows && j < self.cols);
62     self.data[i * self.cols + j]
63 }
64
65 /// Calculate and return the hash of the matrix data
66 fn compute_hash(&self) -> u64 {
67     let mut hasher = DefaultHasher::new();
68     self.hash(&mut hasher);
69     hasher.finish()
70 }
71
72 /// Check the hash of the matrix data against the stored hash
73 /// If the stored hash is 0, initialize it with the current hash
74 fn check_hash(&mut self) {
75     print!("Hash check: ");
76     let new_hash = self.compute_hash();
77     if self.data_hash == 0 {
78         println!("Init");
79         self.data_hash = new_hash;
80     } else if self.data_hash == new_hash {
81         println!("Ok");
82     } else {
83         println!("BAD");
84     }
85 }

```

```

86 }
87
88 // MatrixMultiplier have to live at least as long as A and B
89 struct MatrixMultiplier {
90     A: Matrix, // First matrix
91     B: Matrix, // Second matrix
92     C: Matrix, // Result matrix
93 }
94
95 impl MatrixMultiplier {
96     fn new(A: Matrix, B: Matrix) -> Self {
97         assert!(A.cols == B.rows);
98         let C = Matrix::new(A.rows, B.cols);
99         MatrixMultiplier { A, B, C }
100     }
101
102     /// Row major order multiplication
103     fn multiply_standard(&mut self) {
104         println!("Matrix {}x{} standard ...", self.A.rows, self.B.cols);
105         Self::_with_timer(|| {
106             for A_row in 0..self.A.rows {
107                 for B_col in 0..self.B.cols {
108                     let mut sum = 0;
109                     for k in 0..self.A.cols {
110                         sum += self.A.get(A_row, k) * self.B.get(k,
111 B_col);
112                     }
113                     self.C.set(A_row, B_col, sum);
114                 }
115             });
116         self.C.check_hash();
117     }
118
119     /// Column major order multiplication
120     fn multiply_cols(&mut self) {
121         println!("Matrix {}x{} column-wise ...", self.A.rows, self.B.
122 cols);
123         Self::_with_timer(|| {
124             for B_col in 0..self.B.cols {
125                 for A_row in 0..self.A.rows {
126                     let mut sum = 0;
127                     for k in 0..self.B.rows {
128                         sum += self.A.get(A_row, k) * self.B.get(k,
129 B_col);

```

```

129         }
130         self.C.set(A_row, B_col, sum);
131     }
132 }
133 });
134 self.C.check_hash();
135 }
136
137 /// Concurrent multiplication using num_threads threads
138 /// Each thread computes a chunk of elements of the result matrix
139 /// Resulting matrix size must be divisible by num_threads
140 fn multiply_concurrent(&mut self, num_threads: usize) {
141     // Implement concurrent multiplication
142     // let (chunks, remainder) = self.C.data.as_chunks_mut::<3>();
143     println!(
144         "Matrix {}x{} concurrent with {} threads ...",
145         self.A.rows, self.B.cols, num_threads
146     );
147     assert!(num_threads > 0);
148     assert!(num_threads <= self.C.data.len()); // No more threads
149     than elements
150     assert!(self.C.data.len() % num_threads == 0); // For simplicity
151     require divisibility
152     // Then all elements are equally spread per each thread
153     let chunk_size = self.C.data.len() / num_threads;
154
155     // Channel to send computed cell values from threads to main
156     thread
157     struct CellValue {
158         i: usize,
159         j: usize,
160         value: Elem,
161     }
162     let (tx, rx) = channel::<CellValue>();
163
164     // Clone A and B into Arcs to share between threads
165     let A: Arc<Matrix> = Arc::new(self.A.clone());
166     let B: Arc<Matrix> = Arc::new(self.B.clone());
167     let mut handles = vec![];
168
169     Self::_with_timer(|| {
170         for thread_id in 0..num_threads {
171             let tx_clone = tx.clone();
172             let A_clone = A.clone();
173             let B_clone = B.clone();

```

```

172 // Determine the chunk of elements this thread will
compute
173 let start_element = thread_id * chunk_size;
174 let end_element = start_element + chunk_size;
175 let handle = thread::spawn(move || {
176     for index in start_element..end_element {
177         // Convert linear index to 2D indices
178         // Note: this is probably not the most efficient
way
179         let i = index / B_clone.cols;
180         let j = index % B_clone.cols;
181
182         let mut sum = 0;
183         // Compute the value for C[i][j]
184         for k in 0..A_clone.cols {
185             sum += A_clone.get(i, k) * B_clone.get(k, j)
;
186         }
187         tx_clone.send(CellValue { i, j, value: sum }).
unwrap();
188     }
189 });
190 handles.push(handle);
191 }
192
193 drop(tx); // Close the sending side
194
195 // Multiple producers, single consumer
196 // But we can have multiple consumers with mutex or crates
like crossbeam
197 for cell in rx.iter() {
198     self.C.set(cell.i, cell.j, cell.value);
199 }
200
201 for handle in handles {
202     handle.join().unwrap();
203 }
204 });
205 self.C.check_hash();
206 }
207
208 /// Wrap a function with a timer that prints elapsed time every 100
ms
209 fn _with_timer<F, R>(f: F) -> R
210 where
211     F: FnOnce() -> R,

```



```

212 {
213     let start = Instant::now();
214     let running = Arc::new(AtomicBool::new(true));
215
216     // Clone the running flag for the timer thread
217     let running_clone = running.clone();
218     // Spawn a timer thread
219     let handle = thread::spawn(move || {
220         while running_clone.load(Ordering::Relaxed) {
221             let elapsed = start.elapsed().as_secs_f32();
222             print!("\rElapsed: {:.2} sec", elapsed);
223             std::io::stdout().flush().unwrap();
224             thread::sleep(Duration::from_millis(100));
225         }
226     });
227
228     // Run the actual function
229     let result = f();
230
231     // Tell the timer to stop
232     running.store(false, Ordering::Relaxed);
233
234     // Wait for timer thread to finish
235     handle.join().unwrap();
236
237     // Final time print
238     println!("\rTook: {:.2} sec", start.elapsed().as_secs_f32());
239
240     result
241 }
242 }
243
244 fn main() {
245     let mut A = Matrix::new(MAX_N, MAX_N);
246     A.fill();
247     let mut B = Matrix::new(MAX_N, MAX_N);
248     B.fill();
249
250     let mut multiplier = MatrixMultiplier::new(A, B);
251     multiplier.multiply_standard();
252     println!();
253     multiplier.multiply_cols();
254     println!();
255     multiplier.multiply_concurrent(2);
256     println!();
257     multiplier.multiply_concurrent(4);

```

```

258     println!();
259     multiplier.multiply_concurrent(8);
260     println!();
261     multiplier.multiply_concurrent(100);
262     println!();
263 }
264
265 /// Unit tests for MatrixMultiplier
266 /// Multiplication of two 3x3 matrices with known result
267 #[cfg(test)]
268 mod tests {
269     use super::*;
270     const a: [Elem; 9] = [1, 2, 3, 4, 5, 6, 7, 8, 9]; // Left matrix
271     const b: [Elem; 9] = [9, 8, 7, 6, 5, 4, 3, 2, 1]; // Right matrix
272     const c: [Elem; 9] = [30, 24, 18, 84, 69, 54, 138, 114, 90]; //
Expected result
273
274     #[test]
275     fn test_matrix_multiplier_by_row() {
276         _with_expected(|multiplier| {
277             multiplier.multiply_standard();
278         });
279     }
280
281     #[test]
282     fn test_matrix_multiplier_by_col() {
283         _with_expected(|multiplier| {
284             multiplier.multiply_cols();
285         });
286     }
287
288     #[test]
289     fn test_matrix_multiplier_concurrent() {
290         _with_expected(|multiplier| {
291             multiplier.multiply_concurrent(3);
292         });
293     }
294
295     /// Helper function to create a MatrixMultiplier with predefined
matrices and check the result
296     fn _with_expected<F>(f: F)
297     where
298         F: FnOnce(&mut MatrixMultiplier),
299     {
300         let mut A = Matrix::new(3, 3);
301         A.data = a.to_vec();

```

```

302         let mut B = Matrix::new(3, 3);
303         B.data = b.to_vec();
304         let mut multiplier = MatrixMultiplier::new(A, B);
305         f(&mut multiplier);
306         assert_eq!(multiplier.C.data, c.to_vec());
307     }
308 }

```

Вывод программы

Программа выводит замеры времени выполнения различных реализаций умножения матриц и проверку корректности результата с помощью хеш-суммы:

Листинг 2: Вывод программы

```

1 Matrix 1000x1000 standard ...
2 Took: 29.84 secsec
3 Hash check: Init
4
5 Matrix 1000x1000 column-wise ...
6 Took: 29.63 secsec
7 Hash check: Ok
8
9 Matrix 1000x1000 concurrent with 2 threads ...
10 Took: 16.72 secsec
11 Hash check: Ok
12
13 Matrix 1000x1000 concurrent with 4 threads ...
14 Took: 8.81 secsec
15 Hash check: Ok
16
17 Matrix 1000x1000 concurrent with 8 threads ...
18 Took: 6.41 secsec
19 Hash check: Ok
20
21 Matrix 1000x1000 concurrent with 100 threads ...
22 Took: 5.67 secsec
23 Hash check: Ok

```

Вывод

В данной лабораторной работе была реализована программа для умножения двух квадратных матриц размером 1000x1000 с использованием различных

подходов: стандартного последовательного алгоритма, алгоритма с обходом по столбцам и параллельного алгоритма с использованием нескольких потоков.

Характеристики компьютера

Листинг 3: Характеристики компьютера

```
1 > inxi --cpu --memory
2 Memory:
3   System RAM: total: 16 GiB available: 14.8 GiB used: 10.48 GiB (70.8%)
4   Array-1: capacity: 64 GiB slots: 4 modules: 4 EC: None
5   Device-1: Channel-A DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
6   Device-2: Channel-B DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
7   Device-3: Channel-C DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
8   Device-4: Channel-D DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
9 CPU:
10  Info: 6-core model: AMD Ryzen 5 7640HS w/ Radeon 760M Graphics bits:
    64
11    type: MT MCP cache: L2: 6 MiB
12  Speed (MHz): avg: 1200 min/max: 400/5017 cores: 1: 1390 2: 1100 3:
    1100
13    4: 1100 5: 1100 6: 1100 7: 1396 8: 1100 9: 1432 10: 1100 11: 1389
    12: 1100
```